# GUÍA DE LABORATORIO

## Construcción asistida por IA de una aplicación web para inventario y flujo de caja

*Antigravity + Google Stitch + Spring Boot + React + H2*

| | |
|---|---|
| Asignatura / área | FUNDAMENTOS SISTEMAS DE INFORMACIÓN |
| Modalidad | Laboratorio guiado, individual o en parejas |
| Duración estimada | 4 a 6 horas de trabajo autónomo, más el tiempo de generación asistida |
| Producto de la práctica | PRD, mockups, especificación técnica y prototipo funcional integrado |

## 1. Introducción

En esta práctica se construirá un prototipo de una aplicación web para una empresa que comercializa suministros informáticos (computadores, portátiles, software, componentes y servidores). El problema central consiste en mejorar el control del inventario, la rotación de productos, la relación con proveedores y la toma de decisiones sobre compras y flujo de caja.

La práctica emplea herramientas de inteligencia artificial generativa como apoyo a distintas etapas del ciclo de desarrollo: planeación mediante un documento de requerimientos de producto (PRD), diseño de interfaces mediante Google Stitch y generación asistida del código con Antigravity. La intención no es aceptar automáticamente lo que produzca la IA, sino dirigirla con requisitos explícitos, revisar sus decisiones y validar la calidad del producto.

La secuencia de trabajo es deliberada: primero se define qué problema se resolverá y qué quedará fuera del alcance; luego se diseñan las vistas; después se especifica la arquitectura y el stack tecnológico; finalmente se genera, ejecuta y prueba el prototipo.

## 2. Objetivos

### 2.1 Objetivo general

Desarrollar un prototipo web para apoyar la gestión de inventario, proveedores, reabastecimiento y seguimiento básico del flujo de caja, utilizando herramientas de IA generativa dentro de un proceso de ingeniería de software guiado por especificaciones.

### 2.2 Objetivos específicos

- Elaborar un PRD en Markdown que describa el problema, los usuarios, el alcance, los requisitos funcionales y no funcionales, los flujos y las prioridades del MVP.
- Generar mockups de las vistas principales a partir del PRD y exportarlos como archivos utilizables en el proyecto.
- Definir una especificación técnica con arquitectura por capas, tecnologías, modelo de datos, API y plan de implementación.
- Generar un backend con Spring Boot y API REST, usando H2 en memoria y datos de prueba, y un frontend con React.
- Ejecutar el prototipo y comprobar que las vistas, filtros y operaciones principales funcionan de forma coherente.

- Evaluar críticamente el código generado por IA, identificando defectos, decisiones cuestionables y mejoras necesarias antes de cualquier despliegue real.

## 3. Resultados de aprendizaje

- Traducir una necesidad de negocio a requisitos verificables.
- Reconocer la utilidad y las limitaciones de un PRD como artefacto de planeación.
- Relacionar mockups, requisitos, arquitectura, API y código.
- Identificar responsabilidades de controladores, servicios y repositorios en una aplicación Spring Boot.
- Aplicar pruebas básicas de interfaz y API, y documentar hallazgos.
- Usar IA generativa con supervisión humana y criterios de calidad.

## 4. Escenario del negocio y alcance

La empresa del caso vende suministros informáticos y necesita cerrar cada mes con mejor visibilidad sobre sus existencias, sus compromisos de pago y la capacidad de realizar nuevos pedidos. También necesita detectar productos con inventario crítico, baja rotación u obsolescencia, y comparar proveedores antes de recomendar una compra.

### 4.1 Módulos funcionales propuestos

| Módulo | Propósito | Ejemplos de funciones |
|---|---|---|
| Tablero de cierre mensual y salud financiera | Presentar un resumen ejecutivo del negocio. | Resumen de métricas; capital inmovilizado; capacidad de reorden; indicadores de cierre. |
| Inventario y catálogo tecnológico | Consultar existencias, rotación y riesgo de obsolescencia. | Listado de productos; stock actual/mínimo/ideal; filtros de productos críticos; productos con más de 90 días. |
| Proveedores y órdenes sugeridas | Apoyar la selección de proveedor y el reabastecimiento. | Listado/gestión de proveedores; comparación de costos; sugerencias de orden de compra. |

### 4.2 Fuera del alcance inicial

- Facturación electrónica integrada con entidades tributarias.
- Contabilidad completa o sustitución de un sistema contable.
- Comercio electrónico B2C y pasarelas de pago.
- Módulo de nómina y prestaciones/parafiscales.
- Autenticación y autorización de usuarios en la primera versión del prototipo.

Estas exclusiones son deliberadas para mantener acotado el MVP. Si se decide incorporarlas, se deben modificar el PRD, la especificación técnica y el plan de trabajo.

## 5. Herramientas y requisitos previos

| Herramienta | Uso en la práctica | Verificación previa |
|---|---|---|
| Google Antigravity IDE (aplicación independiente) | Entorno de desarrollo con agentes de IA para planear, generar y revisar archivos/código. | Instalada y con sesión iniciada. En la descarga, elegir la versión independiente, no solo una extensión de otro IDE. |
| Google Stitch | Generación de propuestas visuales/mockups para interfaces web. | Acceso disponible mediante una cuenta compatible. |
| Java JDK | Compilar y ejecutar el backend | Instalar una versión compatible |

| | | |
|---|---|---|
| | Spring Boot. | con el proyecto que proponga la especificación; en la demostración se utilizó Java 17. |
| Maven / Maven Wrapper | Construcción y ejecución del backend. | Verificar que el proyecto incluya mvnw o mvnw.cmd. |
| Node.js y npm | Instalar dependencias y ejecutar el frontend React. | Verificar con node -v y npm -v. |
| Navegador web | Validar las vistas y los flujos del prototipo. | Chrome, Edge o Firefox actualizado. |
| Git (recomendado) | Control de versiones y registro de cambios. | Repositorio local para conservar versiones del trabajo. |

Nota: las versiones exactas de Spring Boot, React, Node y las dependencias deben quedar registradas en la especificación técnica generada para cada equipo. No se deben asumir versiones distintas a las que el proyecto realmente configure.

# 6. Procedimiento

## Fase 1. Preparación del entorno

1. Instale Google Antigravity IDE como aplicación independiente y ábrala.
2. Inicie sesión y cree una carpeta de trabajo para el proyecto, por ejemplo inventario-flujo-caja.
3. Compruebe que Java JDK, Maven (o Maven Wrapper), Node.js y npm estén instalados. Registre las versiones en el informe.
4. Cree una estructura inicial de carpetas. La organización final podrá cambiar según lo que genere el agente.

```
inventario-flujo-caja/PRD.md

                    design.md (opcional)

mockups/(exportación de Stitch)SPEC-TECNICA.md

backend/

frontend/
```

## Fase 2. Definición del PRD con el agente

Antes de pedir código, se debe construir un documento de requerimientos de producto (PRD). Este artefacto reduce la ambigüedad y permite evaluar si la solución generada corresponde a la necesidad del negocio.

En Antigravity, describa el contexto del negocio y solicite una propuesta de PRD. Puede utilizar el siguiente prompt como punto de partida.  Recuerde ejecutarlo en modo PLAN

```
Eres un experto en desarrollo de Producto. Quiero construir una solución que me ayude a resolver este problema: "Como dueño de un negocio que vende equipos de computo (computadores, accesorios, software, servidores, UPS, etc), cuando hago el cierre de mes, quiero ver los gastos, ingresos, inventario y demás de mi negocio, para poder decidir si hago un nuevo pedido a mis proveedores." Ayudame a definir un "Product Requirement Document" con detalles funcionales sobre la experiencia del usuario, el comportamiento del usuario, los objetivos del producto, el espacio de la solución (limitalo a la gestión de las finanzas del almacen, inventario y proveedores), features claves, con flujos principales.
```

Revise el documento propuesto. No acepte automáticamente sus afirmaciones; ajuste las reglas de negocio, los términos y las prioridades.

### 6.2.1 Métricas de éxito de referencia

- Reducir el tiempo requerido para consolidar el cierre mensual.
- Reducir los quiebres de stock en productos de alta demanda.
- Mejorar la precisión del flujo de caja y evitar pagos retrasados por pedidos imprevistos.
- Medir la proporción de órdenes de compra que se originan en recomendaciones del sistema.

#### 6.2.2 Verificación mínima del PRD

- Cada requisito debe ser comprensible y verificable.
- El alcance debe diferenciar lo incluido y lo excluido.
- Los módulos deben corresponder a las necesidades del negocio.
- Los flujos deben tener inicio, decisiones y resultados claros.
- Los requisitos no funcionales deben incluir criterios comprobables.
- La priorización debe separar lo imprescindible del MVP de las mejoras futuras.

Cuando el PRD esté aprobado por el equipo, solicite guardarlo en el directorio raíz como PRD.md:

```
Genera el PRD definitivo en formato Markdown y guárdalo como PRD.md en el directorio raíz del proyecto. Conserva las decisiones aprobadas y no agregues funcionalidades que no estén en el alcance.
```

## Fase 3. Diseño de interfaces en Google Stitch

5. Abra Google Stitch e inicie sesión.
6. Seleccione el tipo de proyecto para interfaces web, no la opción de aplicación móvil/tablet.
7. Seleccione un modelo o modo de generación que priorice el equilibrio entre rapidez y calidad, según las opciones disponibles en la herramienta.
8. Defina una paleta y estilo visual. Si se requiere mayor consistencia, prepare un archivo design.md con colores, tipografía, espaciado, botones, formularios, estados, mensajes y reglas de distribución.
9. Cargue el archivo PRD.md como referencia para que las vistas respondan a requisitos concretos.
10. Solicite las vistas principales del sistema y revise las propuestas generadas.

```
Con base en el PRD adjunto, diseña las vistas principales de una aplicación web empresarial para gestión de inventario y flujo de caja. Incluye un dashboard, inventario/catálogo tecnológico, productos críticos o con más de 90 días, gestión de proveedores y órdenes de compra sugeridas.

La interfaz debe ser profesional, clara, usable, accesible y consistente. Prioriza jerarquía visual, lectura rápida de KPI, filtros comprensibles, tablas legibles, estados vacíos, errores y confirmaciones.
No añadas módulos que estén fuera del alcance del PRD.
```

Si el primer resultado no es adecuado, refine el prompt con observaciones específicas: distribución, contraste, jerarquía de indicadores, etiquetas, tablas, filtros o navegación. Las vistas generadas son prototipos visuales; no garantizan por sí solas que exista lógica funcional.

#### 6.3.1 Exportación de mockups

11. Seleccione las vistas que se van a conservar.
12. Use la opción de exportación disponible y descargue el archivo ZIP.
13. Descomprima el ZIP dentro de la carpeta del proyecto.
14. Conserve la carpeta exportada (por ejemplo, mockups/stitch/) con los HTML y las capturas que produzca la herramienta.
15. Revise que los archivos estén disponibles localmente y que correspondan a las vistas aprobadas.

Evidencia 3: capturas de las vistas y estructura de la carpeta exportada.

## Fase 4. Especificación técnica

Vuelva a Antigravity y cambie el enfoque del agente a líder técnico/arquitecto de software. Indique explícitamente la ruta del PRD y la carpeta de mockups. Solicite una especificación que describa cómo se implementará la solución, antes de aprobar la generación del código.

```
Eres un ingeniero sénior líder técnico, necesito que revises el @docs/PRD-cierre-mes-almacen.md  y los mockups que están en la carpeta  @stitch_gesti_n_de_almac_n_it/ que contienen vistas previas de lo qué queremos construir. Elabora una especificación técnica donde definas las tecnologías a utilizar y todo lo relacionado con el cómo de la implementación de esta solución. Para el Backend usa Spring Boot con API
```

```
Rest, el frontend dejalo con REACTJS.  Por el momento crea 20 datos de prueba con una base de datos H2 In-
memory. No usaremos por el momento autenticación de usuarios.
```

Revise y apruebe la propuesta. Si la arquitectura, las versiones, el modelo de datos o las rutas no son coherentes, pida correcciones antes de autorizar la implementación.

#### 6.4.1 Elementos mínimos de la especificación

- Arquitectura general y responsabilidades de cada componente.
- Entidades principales: productos, proveedores y los registros adicionales necesarios para soportar los requisitos.
- Relaciones y restricciones de datos.
- Capas del backend: controladores REST, servicios de negocio y repositorios.
- Rutas de API, verbos HTTP, parámetros, formatos de respuesta y errores.
- Estructura de componentes, vistas y servicios del frontend.
- Configuración de H2, datos iniciales y plan de pruebas.
- Fases de implementación y criterios de finalización.

## Fase 5. Generación del proyecto con Antigravity

Una vez revisada la especificación, pida al agente que implemente el proyecto. En la demostración se trabaja primero sobre el backend y luego sobre el frontend, integrando las vistas exportadas desde Stitch.

El agente puede pedir permiso para ejecutar comandos o instalar dependencias. Lea cada comando antes de autorizarlo y evite conceder permisos indiscriminados.

#### 6.5.1 Inspección del backend

Revise al menos los siguientes elementos generados:

- Entidad Product/Producto: identificador, código SKU, nombre, marca, categoría/subcategoría, costos, precio de venta, stock actual, mínimo e ideal, días en inventario y referencia al proveedor, según lo definido en la especificación.
- Entidad y relación de Supplier/Proveedor con los productos.
- Repositorios basados en Spring Data JPA y consultas personalizadas justificadas.
- Servicios con lógica de negocio; no trasladar toda la lógica a los controladores.
- Controladores REST con rutas, códigos HTTP y validación apropiados.
- application.properties o application.yml con configuración de H2 y demás propiedades.
- Pruebas y documentación de API.

#### 6.5.2 Inspección del frontend

- Vistas/componentes separados por responsabilidad.
- Servicios de consumo de API separados de la presentación.
- Indicadores del dashboard alimentados por datos del backend y no solo por valores estáticos.
- Filtros que cambian los resultados mostrados.
- Formularios con validaciones y mensajes comprensibles.
- Manejo visible de carga, errores de red y listas vacías.
- Diseño coherente con los mockups aprobados.

## Fase 6. Ejecución local

Los comandos exactos dependen de la estructura generada. Use primero el README y los scripts del proyecto. Como referencia, el flujo suele ser:

### 6.6.1 Backend

```
# Windows (si existe Maven Wrapper)
cd backend
mvnw.cmd spring-boot:run

# Linux/macOS
cd backend
./mvnw spring-boot:run
```

Espere a que el registro indique que la aplicación inició correctamente. Revise el puerto real configurado en application.properties/application.yml. En la demostración se mencionó el puerto 8080 como ejemplo, pero debe confirmarse en el proyecto generado.

Si la consola de H2 está habilitada, use únicamente la ruta y credenciales de desarrollo configuradas en el proyecto. No exponga la consola H2 en un entorno de producción.

### 6.6.2 Frontend

```
cd frontend
npm install
npm run dev
```

Abra en el navegador la URL local que muestre Vite u otra herramienta de desarrollo. No asuma un puerto fijo: utilice el que aparezca en la consola.

### 6.6.3 Verificación de conectividad

16. Consulte la documentación OpenAPI/Swagger si fue configurada.
17. Ejecute al menos una petición GET de lectura a un endpoint real.
18. Compruebe que el frontend consuma el backend y muestre los mismos datos.
19. Revise la consola del navegador y los registros del backend ante errores.
20. Confirme que la persistencia es H2 en memoria y que, al reiniciar, los datos pueden reinicializarse; no la trate como base de datos persistente.

## Fase 7. Pruebas funcionales y refinamiento

Pruebe el sistema como usuario final y registre los resultados. Si algo no funciona, describa el comportamiento observado, el esperado y los pasos para reproducirlo antes de pedir una corrección al agente.

| ID | Prueba | Resultado esperado |
|---|---|---|
| P01 | Abrir el dashboard | Se muestran indicadores y datos coherentes con las respuestas de la API. |
| P02 | Consultar catálogo de productos | La tabla muestra registros del backend con campos legibles. |
| P03 | Filtrar productos críticos | El filtro devuelve productos que cumplen la condición documentada. |
| P04 | Filtrar productos con más de 90 días | Solo aparecen los productos que cumplen el criterio de antigüedad. |
| P05 | Consultar proveedores | Se presenta el listado de proveedores y sus datos relevantes. |
| P06 | Generar/ver una sugerencia de compra | La recomendación se basa en stock, rotación y reglas definidas; no solo en texto estático. |

| | | |
|---|---|---|
| P07 | Probar datos vacíos o error de API | La interfaz presenta un mensaje útil y no se bloquea. |
| P08 | Validar formularios | Los campos obligatorios y valores inválidos se gestionan correctamente. |
| P09 | Recargar y reiniciar | Se documenta el comportamiento de los datos de H2 en memoria. |
| P10 | Revisar diseño adaptable | La interfaz conserva legibilidad en diferentes tamaños de ventana. |

Para cada fallo, cree un registro con: ID, pasos para reproducir, resultado actual, resultado esperado, captura/log, hipótesis y corrección aplicada.

### Fase 8. Revisión de calidad del código generado

La generación asistida no reemplaza la responsabilidad de ingeniería. Revise manualmente y, cuando sea posible, apoye la revisión con pruebas automatizadas o análisis estático.

- Trazabilidad: cada función importante debe corresponder a un requisito del PRD.
- Separación de responsabilidades: controladores, servicios, repositorios y componentes tienen responsabilidades claras.
- Validación: entradas y reglas de negocio se validan en el servidor, no únicamente en el frontend.
- Manejo de errores: respuestas y mensajes no exponen trazas o información sensible.
- Calidad de datos: las métricas y los totales se calculan con reglas explícitas y unidades consistentes.
- Seguridad: no se incluyen secretos en el repositorio; el prototipo sin autenticación no es apto para datos reales.
- Mantenibilidad: nombres, estructura, duplicación, complejidad y documentación.
- Pruebas: se distinguen pruebas ejecutadas de las sugeridas por el agente.
- Deuda técnica: documente atajos y elementos que deben resolverse antes de producción.

## 7. Entregables . EJERCICIO DE ENTREGA OPCIONAL FECHA 13 de NOVIEMBRE DE 2026

**EJERCICIO PROPUESTO: Defina un caso de estudio como por ejemplo la administración de activos de una bodega, la gestión de facturación de una empresa, la gestión de nomina de una PYME, o la gestión de ventas de un supermercado. Luego defina un plan de producto, su PRD y su implementación completo. Debe evidenciar los siguientes elementos en el informe**

- Carpeta del proyecto con código fuente del backend y frontend.
- PRD.md revisado y aprobado por el equipo.
- Carpeta de mockups exportados desde Stitch, incluyendo capturas disponibles.
- SPEC-TECNICA.md con arquitectura, modelo de datos, API y plan de pruebas.
- README.md con requisitos, versiones, instrucciones de ejecución y configuración.
- Matriz de pruebas diligenciada con evidencias y resultados reales.
- Video corto o demostración en vivo del flujo principal, si lo solicita el docente.

## 8. Recomendaciones y advertencias

- No solicite a la IA “crear toda la aplicación” sin antes definir alcance, requisitos y criterios de aceptación.
- Verifique el contenido de cada archivo generado y las versiones que realmente utiliza el proyecto.
- No ejecute comandos destructivos ni instale dependencias sin comprender su propósito.
- No use credenciales, datos personales ni información confidencial en los prompts.
- Los datos generados son ficticios y solo deben usarse para pruebas.
- La ausencia de autenticación y el uso de H2 en memoria son decisiones temporales del prototipo; no constituyen una configuración de producción.
- Si una prueba no se ejecutó, márquela como pendiente y no como aprobada.
- Las pantallas, los nombres de archivos y las opciones de las herramientas pueden cambiar con sus actualizaciones; documente cualquier diferencia.

## 9. Referencias y recursos

- Google Antigravity: https://antigravity.google/
- Google Stitch: https://stitch.withgoogle.com/
- Spring Boot — documentación oficial: https://docs.spring.io/spring-boot/
- Spring Data JPA — documentación oficial: https://spring.io/projects/spring-data-jpa
- React — documentación oficial: https://react.dev/
- H2 Database — sitio oficial: https://www.h2database.com/
- Mermaid — diagramas como código: https://mermaid.js.org/

## Anexo A. Checklist de entrega

- ☐ PRD.md revisado y consistente con el problema del negocio.
- ☐ Mockups exportados y guardados en la carpeta del proyecto.
- ☐ SPEC-TECNICA.md actualizado con decisiones aprobadas.
- ☐ Backend compila y arranca sin errores bloqueantes.
- ☐ Frontend instala dependencias y arranca correctamente.
- ☐ Al menos una ruta REST fue probada directamente.
- ☐ Los datos que aparecen en la interfaz proceden del backend.
- ☐ Matriz de pruebas completada con evidencia.
- ☐ Hallazgos de calidad y limitaciones documentados.
- ☐ README contiene instrucciones reproducibles para ejecutar el proyecto.